

18 · Guía para un nuevo desarrollador

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **2282a33**

Fecha de generacion: **2026-08-27**

Generado a partir de 18-new-developer-guide.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Antes de nada
2. Día 1 — Poner en marcha el proyecto
3. Día 1 — Lectura mínima
4. Día 2 — Seguir un flujo completo
5. Día 3 — Cómo está organizado el repositorio
6. Dónde añadir cosas
7. Cómo escribir pruebas aquí
8. Convenciones que hay que respetar
9. Partes que requieren especial cuidado
10. Errores que probablemente cometerás
11. Primeras tareas sugeridas
12. Itinerario de una semana
13. A quién preguntar y dónde mirar

Itinerario de incorporación: qué leer, en qué orden, qué ejecutar y qué tocar primero. Pensado para llegar a hacer un cambio útil en tu primer día y entender el sistema entero en tu primera semana.

1. Antes de nada

Tres cosas que conviene saber desde el minuto uno, porque explican casi todas las decisiones que verás en el código:

1. **El producto detecta cambios, no amenazas.** No hay firmas ni catálogos de malware. Todo se apoya en comparar el estado actual con una foto guardada.
2. **La evidencia es parte del producto.** Nunca se muestra un veredicto sin el dato crudo que lo respalda. Si añades una comprobación, tiene que traer su evidencia.
3. **Lo que no se sabe se declara.** Un control cuyo estado no se pudo determinar se pinta de amarillo, no de verde. Una sección sin permisos dice por qué está vacía.

Si un cambio tuyo rompe alguna de las tres, romperá también algún test: hay pruebas dedicadas precisamente a fijar esos criterios.

2. Día 1 — Poner en marcha el proyecto

```
git clone https://github.com/vladimiracunadev-create/rootcause-macos-inspector.git
cd rootcause-macos-inspector
./scripts/verify-environment.sh      # ¿tengo todo?
cargo build --release                # 3-8 minutos la primera vez
./target/release/rootcause status   # el producto, en consola
./target/release/rootcause          # el producto, con interfaz
```

Después, dedica media hora a **usarlo**: pulsa por las doce secciones, mira qué muestra cada una en tu propio equipo, exporta una captura (■E) y abre el JSON. Entender qué produce el sistema hace que el código se lea solo.

Ejecuta también la validación completa, para saber cómo se ve el verde:

```
./scripts/ci-local.sh
```

3. Día 1 — Lectura mínima

Orden	Documento	Por qué
1	README.md	La tesis del producto en cinco minutos
2	01 · Descripción general	Qué hace y para quién
3	03 · Arquitectura	Las capas y por qué están separadas así
4	04 · Mapa del código	Dónde está cada cosa

Con eso ya puedes moverte. El resto se lee cuando toque.

4. Día 2 — Seguir un flujo completo

La mejor forma de entender el sistema es seguir **una** captura de punta a punta. Abre estos archivos en este orden y lee solo lo que se indica:

Paso	Archivo	Qué leer
1	src/main.rs	Entero: son 119 líneas y define el arranque

2	<code>src/services/inspector.rs</code>	Solo <code>collect_snapshot</code> : es la secuencia completa
3	<code>src/services/macros.rs</code>	<code>run_capture</code> , <code>run_combined</code> y <code>process_details</code>
4	<code>src/services/launchd.rs</code>	<code>scan_persistence</code> , <code>parse_launch_plist</code> y <code>classify_entry</code>
5	<code>src/services/baseline.rs</code>	Entero: son 217 líneas y es el corazón del producto
6	<code>src/services/rules.rs</code>	<code>classify_process</code> , <code>build_alerts</code> y <code>derive_incident</code>
7	<code>src/services/persistence.rs</code>	<code>ensure_schema</code> y <code>replace_baseline</code>
8	<code>src/app.rs</code>	Solo <code>spawn_worker</code> y <code>update</code> : el resto es dibujado

Acompaña la lectura con [06 · Explicación profunda](#), que explica las decisiones no evidentes de cada uno de esos puntos.

Ejercicio útil: pon un `dbg!` en `collect_snapshot` después del bloque de anomalías, ejecuta `cargo run -- status` y observa qué se genera en tu equipo. Recuerda quitarlo antes de commitear: `clippy` con `-D warnings` no lo permite.

5. Día 3 — Cómo está organizado el repositorio

```
src/
■■■ main.rs      ← arranque: decide CLI o GUI
■■■ cli.rs       ← 19 comandos
■■■ app.rs       ← interfaz (el archivo más grande)
■■■ models.rs    ← el vocabulario común; no importa a nadie
■■■ config.rs    ← configuración y valores por defecto
■■■ i18n.rs      ← traducción ES/EN
■■■ meta.rs      ← constantes del producto
■■■ services/
    ■■■ inspector.rs ← orquesta la captura ← empieza aquí
    ■■■ macros.rs    ← ÚNICO punto que habla con el sistema
    ■■■ launchd.rs · security.rs · tcc.rs · network.rs · netscan.rs · temp_scan.rs
    ■               ← un recolector por superficie
    ■■■ rules.rs · anomaly.rs · baseline.rs
    ■               ← deciden qué significan los datos
    ■■■ persistence.rs ← SQLite
    ■■■ resilience.rs ← salud del propio agente
    ■■■ report.rs     ← reporte Markdown
    ■■■ ai.rs         ← adaptador opcional
```

Regla de oro para orientarte: **si toca el sistema operativo, va en `macros.rs`; si decide, va en `rules.rs` o `anomaly.rs`; si solo transforma datos, va en el recolector de su superficie.**

6. Dónde añadir cosas

6.1 Una comprobación nueva en una superficie existente

Ejemplo: añadir un control de seguridad.

1. Escribe una función privada en `src/services/security.rs` que ejecute el comando con `macros::run_combined` y devuelva un `SecurityControl`.
2. Añádela al `vec![...]` de `scan_controls`.
3. Rellena `evidence` con la primera línea útil de la salida y `explanation` con qué significa y qué hacer si está apagado.

4. Usa `severity_for(enabled, known, cuando_apagado)`: **no calcules la severidad a mano**, porque esa función codifica la regla de que «desconocido» es amarillo.
5. Añade un test con una salida literal del comando.

No hace falta tocar nada más: la baseline, las alertas y la interfaz lo recogen solas, porque `control_watch_items` recorre la lista completa.

6.2 Una heurística nueva

1. Añade sus parámetros a `AnomalyConfig` en `src/config.rs`, con su función `default_*`.
2. Impleméntala dentro del bucle de `AnomalyTracker::analyze`, usando `process_event` para construir el evento.
3. Elige un `kind` en minúsculas con guiones y no reutilices uno existente.
4. Decide si necesita racha (estado en `ProcessHistory`) o si es inmediata.
5. Añade al menos dos tests: uno que dispare y **uno que compruebe que no dispara con software normal**. El segundo importa más que el primero.

6.3 Una superficie vigilada nueva

Es el cambio más grande y el que mejor muestra la arquitectura:

1. Modelo en `src/models.rs` con `Serialize/Deserialize` y `#[serde(default)]` en los campos nuevos.
2. Recolector en `src/services/<superficie>.rs` con una función `scan()` y una `*_watch_items()`.
3. Si necesita el sistema, añade la llamada a `src/services/macros.rs`, nunca directamente.
4. `SurfaceSpec` en `src/services/baseline.rs` con sus textos y su riesgo.
5. Campo en `SystemSnapshot` y llamada en `collect_snapshot`, junto con una función `detect_<superficie>_changes` siguiendo el patrón de las cuatro existentes.
6. Interruptor `watch_<superficie>` en `AnomalyConfig`.
7. Sección en `src/app.rs` y comando en `src/cli.rs`.
8. Tests del parser y de la clasificación.

6.4 Un comando nuevo del CLI

1. Añade la rama al `match` de `cli::run`.
2. Escribe `cmd_<nombre>` siguiendo el patrón: crear servicio, obtener datos, `--json` si se pide, salida tabulada si no.
3. Documenta el comando en `print_help`, en `docs/COMMANDS.md` y en [05 · Referencia técnica](#).
4. Usa `truncate` para las columnas: hay un test que protege el corte con acentos.

7. Cómo escribir pruebas aquí

- Van **al final del archivo**, en un módulo `#[cfg(test)] mod tests`.
- Nombres en español, descriptivos y en forma de afirmación: `un_pico_de_cpu_no_dispara_nada`, no `test_cpu`.
- Los datos de muestra son constantes literales dentro del test; no hay fixtures.
- Prueba **el criterio**, no la implementación: si mañana cambia la forma de calcular el puntaje pero un compilador legítimo sigue sin pintarse de rojo, el test debe seguir en verde.

8. Convenciones que hay que respetar

Ámbito	Convención
Formato	<code>cargo fmt</code> ; ancho 100, saltos Unix. La CI lo verifica

Lints	<code>clippy - D warnings</code> . Nada de <code>#[allow]</code> sin comentario que lo justifique
Idioma del código	Identificadores en inglés; comentarios, mensajes y documentación en español
Comentarios	Explican por qué , no qué. Si el comentario repite la línea, sobra
Documentación de módulo	Todo archivo empieza con <code>///!</code> explicando su papel
Documentación pública	Todo elemento <code>pub</code> lleva <code>///</code> . Actualmente 232 de 232
Errores	<code>anyhow::Result</code> con <code>.context(...)</code> en las rutas que ve el usuario
Rutas del sistema	Siempre absolutas (<code>/usr/sbin/lsof</code>), nunca por <code>PATH</code>
Markdown	Líneas de hasta 100 caracteres; <code>markdownlint-cli2</code> lo verifica
Commits	Mensaje en español, imperativo, explicando el porqué

9. Partes que requieren especial cuidado

Zona	Por qué	Qué hacer antes de tocarla
<code>baseline.rs</code> y <code>persistence.rs</code>	Un error aquí silencia detecciones sin que nadie lo note	Entender la propiedad «pegajosa» y la siembra silenciosa
<code>persistence_entry_key</code>	Cambiar la clave invalida todas las baselines existentes de los usuarios	Considerar la migración
<code>can_terminate_process</code> y la lista de protegidos	Aflojarla permite matar procesos que dejan el equipo inutilizable	No la hagas configurable
<code>clean_user_caches</code>	Es el único código que borra archivos	Mantener las tres salvaguardas y el <code>dry_run</code> por defecto
<code>ai.rs</code>	Es la única salida de red	No añadas datos al payload sin actualizar la política de privacidad y su test
<code>classify_*</code>	Los puntajes son el producto	Cambiar un peso cambia el comportamiento para todos los usuarios; hazlo con un test que lo justifique
<code>macos.rs</code>	Un cambio de formato en macOS rompe el parser	Preferir «desconocido» a adivinar
Esquema SQLite	No hay migraciones	Añadir una columna a una tabla existente no funcionará en instalaciones ya creadas

10. Errores que probablemente cometerás

Todos evitables, todos vistos en el código que ya está:

1. **Llamar a `Command` fuera de `macos.rs`.** Rompe la capa de adaptador y hace el módulo imposible de probar.
2. **Calcular una severidad a mano** en vez de usar el helper de su superficie.
3. **Cortar cadenas por bytes.** Los textos están en español; usa `truncate`, que corta por caracteres.
4. **Añadir un `unwrap()` sobre una salida del sistema.** Todo lo que viene de fuera puede faltar o cambiar de forma.

5. **Actualizar la baseline dentro del diff.** Solo se actualiza al sembrar y al aceptar explícitamente.
6. **Añadir una dependencia.** Es el proyecto de un producto que presume de tener diez; justifica muy bien cualquier crate nuevo.
7. **Olvidar que hay dos ediciones.** Si tocas algo compartido, comprueba también `cargo build --no-default-features`.

11. Primeras tareas sugeridas

Ordenadas de menor a mayor dificultad. Las tres primeras se pueden hacer el primer día.

#	Tarea	Qué aprendes	Dificultad
1	Añadir tres prefijos OUI nuevos a <code>netscan::OUI_TABLE</code> con su test	Estructura de un recolector y de sus tests	Muy baja
2	Añadir un servicio TCC más a <code>service_label</code>	Cómo se traduce el vocabulario del sistema	Muy baja
3	Extraer <code>expand_home</code> y <code>bytes_to_mb</code> a un módulo común (R-14)	Organización del proyecto	Baja
4	Sustituir <code>static mut ACTIVE_DARK</code> por <code>AtomicBool</code> (R-13)	El modelo de estado de la interfaz	Baja
5	Implementar <code>notification_cooldown_secs</code> (R-05)	Estado entre capturas en <code>InspectorService</code>	Media
6	Validar que los umbrales críticos superen a los de aviso (R-11)	Carga de configuración y advertencias	Media
7	Escribir los tests de <code>PersistenceStore</code> sobre una base temporal (R-18)	Toda la capa de persistencia	Media
8	Añadir <code>PRAGMA user_version</code> y migraciones (R-06)	Evolución del esquema	Media-alta
9	Implementar el deshacer de baseline (R-01)	El núcleo del producto	Alta

Las referencias `R-xx` remiten a [15 · Riesgos y deuda técnica](#), donde cada una está descrita con su recomendación.

12. Itinerario de una semana

Día	Objetivo	Entregable
1	Entorno listo y producto ejecutándose; leer README, 01, 03 y 04	<code>ci-local.sh</code> en verde en tu equipo
2	Seguir una captura de punta a punta con el documento 06 al lado	Poder explicar qué hace <code>collect_snapshot</code>
3	Entender baseline y persistencia (documentos 07 y 08)	Poder explicar por qué un cambio es «pegajoso»
4	Primera tarea (1, 2 o 3 de la lista) con su test	Pull request pequeño y en verde

5	Leer 10, 11 y 12; ejecutar el generador de PDF	Segunda tarea, de dificultad media
---	--	------------------------------------

13. A quién preguntar y dónde mirar

Duda	Fuente
«¿Por qué está hecho así?»	El comentario junto al código; casi siempre está respondido ahí
«¿Qué hace este comando?»	docs/COMMANDS.md y 05
«¿Qué significa este término?»	16 · Glosario
«¿Esto es un fallo conocido?»	15 · Riesgos
«¿Cómo se prueba esto?»	12 · Pruebas
«¿Dónde vive este dato?»	07 · Base de datos y 19 · Trazabilidad
«¿Por qué falla mi entorno?»	14 · Solución de problemas

Siguiente lectura recomendada: [19 · Matriz de trazabilidad](#).